

Software Project Management Plan (SPMP)

Car Rental Management System

Course: Software Engineering / Project Management

Supervision: Department of Computer Science, Quaid-i-Azam University, Islamabad

Supervised by: Dr. Umer Rashid

Standard Compliance: ISO/IEC/IEEE 16326:2019 (formerly IEEE Std 1058-1998)

Signature Page

Name	Role	Signature
Fazal-e-Rabbi (04072414035)	Project Manager / Architecture Lead	
Samiullah Saleem (04072414027)	Backend Developer / Database Engineer	
Hafiz Mustafa Saleem (04072414005)	Full-Stack Developer / API Integration Specialist	
Muneeb ur Rehman (04072414010)	Quality Assurance / UI/UX Specialist	

Preface

This document establishes the Software Project Management Plan (SPMP) for the Car Rental Management System. Structured in strict accordance with IEEE Std 1058-1998 (now integrated into ISO/IEC/IEEE 16326:2019), this plan sets forth the technical lifecycle processes, Work Breakdown Structure (WBS), project schedule baseline, resource assignments, and risk mitigation protocols required to take the software from initial requirement analysis to final production verification and sign-off.

Table of Contents

- 1. Project Overview
 - 1.1 Project Summary
 - 1.2 Purpose, Scope, and Objectives
 - 1.3 Assumptions and Constraints
 - 1.4 Project Deliverables
 - 1.5 Schedule Summary
- 2. References
- 3. Definitions and Acronyms
- 4. Project Context and Technical Process Plans
 - 4.1 Process Model
 - 4.2 Methods, Tools, and Techniques
 - 4.3 Product Acceptance Plan
 - 4.4 Product Acceptance Methods
- 5. Project Work and Schedule Allocation
 - 5.1 Project Work Plan (WBS Structure)

- 5.2 Work Activities
- 5.3 Schedule Allocation
- 5.4 Resource Allocation

6. Supporting Process Plans

- 6.1 Risk Management (Identification & Analysis)
- 6.2 Risk Mitigation Strategies
- 6.3 Risk Monitoring and Control

1. Project Overview

1.1 Project Summary

Traditional vehicle rental businesses depend heavily on manual, paper-bound ledgers, unstandardized telephone confirmations, manual vehicle status tracking, and offline cash accounting. These outdated operations introduce scheduling conflicts, administrative overhead, human error in billing calculations, and lack of real-time vehicle fleet tracking.

The Car Rental Management System is a standalone, web-based platform structured around a three-tier architecture that automates fleet inventory management, dynamic availability queries, rental cost calculations, digital payment transactions, and reservation workflows.

1.2 Purpose, Scope, and Objectives

1.2.1 Purpose

The purpose of the system is to establish an automated, centralized digital environment connecting rental agencies, fleet administrators, and customers. The system aims to:

- Digitize and automate vehicle discovery, date-based booking, and inventory status tracking.
- Provide role-based access control (RBAC) to isolate administrative controls from consumer workflows.
- Ensure transparent billing via automated calculation engines and external digital payment gateways.
- Provide fleet oversight through analytical reporting on revenue, vehicle utilization, and maintenance scheduling.

1.2.2 Scope

- Included Capabilities:
- Guest/Public Access: Browse public vehicle listings, filter by brand/category, and view contact and agency details.
- Customer Module: Account registration (validating 11-digit mobile contacts and secure credentials), authenticated login and session handling, vehicle discovery (location, date range, car type), reservation placement, automated cost calculation, digital payment processing (credit card/Stripe/PayPal), and password recovery/updates.
- Administrative Module: Vehicle catalog and brand management (CRUD operations for models, number plates, and pricing), live availability/maintenance toggles, booking approval/rejection oversight, customer profile inspections, and revenue utilization analytics.
- Exclusions: Excludes automotive maintenance hardware telemetry (e.g., physical OBD-II dongle integration), roadside physical mechanical dispatch, and peer-to-peer (P2P) host listings.

1.2.3 Objectives

- Achieve low-latency database queries, keeping vehicle search response times under 1.5 seconds.
- Maintain 99.9% web application uptime to allow continuous booking and status verification.
- Maintain end-to-end data security by encrypting sensitive records and user credentials with SHA-256 or equivalent standards.
- Complete system implementation, test validation, and the final academic presentation within the semester timeline.

1.3 Assumptions and Constraints

Assumptions

- Users have access to an active internet connection and a standard web browser on desktop or mobile devices.
- External third-party APIs (Stripe/PayPal for payment processing, Google Maps API for spatial tracking) maintain operational uptime and accessible sandbox endpoints.
- Fleet managers consistently update vehicle maintenance records and operational statuses in the database.
- Customers provide verifiable 11-digit phone numbers and valid email addresses during account creation.

Constraints

- Architecture Constraint: Must follow a clean three-tier web architecture (Client, Server, Database).
- Technology Stack: Frontend must run on standard HTML5, CSS3 (Bootstrap), and JavaScript; the backend must utilize Node.js or PHP; persistence is strictly managed through MySQL.
- Security & Integrity: Passwords and identification numbers must not be stored in plain text; referential integrity must be enforced via foreign keys across Users, Cars, Bookings, and Payments tables.
- Schedule Deadline: All engineering deliverables—from requirements specification to deployment and defense—must conclude by the academic submission window.

1.4 Project Deliverables

- Software Project Management Plan (SPMP): This operational planning document adhering to ISO/IEC/IEEE 16326.
- ProjectLibre Gantt Chart & Work Breakdown Structure: Detailed baseline project schedule and resource leveling plans.
- Software Requirements Specification (SRS): IEEE Std 830-1998 compliant document specifying functional, non-functional, ERD, and use case requirements.
- Software Design Document (SDD): Complete database schema, normalized data dictionaries, component interfaces, and activity/flow charts.
- Interactive Wireframes & Functional Prototype: Working prototype demonstrating registration, car search, and administrative dashboard layouts.
- Source Code Repository: Production-ready source code repository on GitHub containing backend services, frontend layouts, and database migrations.
- Quality Assurance & Test Suite: Documented unit testing suites, integration tests, and user acceptance test (UAT) execution logs.
- Project Defense & System Demonstration: Live functional demonstration of the car booking and fleet administration workflows.

1.5 Schedule Summary

Task or Phase	Target Window	Deliverables
---------------	---------------	--------------

Phase 1: Project Initiation & Planning	Week 1 – Week 2	SPMP Document, ProjectLibre Schedule Baseline, Toolchain Setup
Phase 2: Requirements Analysis	Week 3 – Week 5	Completed SRS Document, Use Case Models, Domain Analysis
Phase 3: System & Database Design	Week 6 – Week 8	Normalized MySQL Schema, ERD Diagrams, Flowcharts, UI Wireframes
Phase 4: Core Implementation (Sprints 1–4)	Week 9 – Week 12	Functional Working Prototype, Fleet Management Module, Booking & Payment APIs
Phase 5: System Verification & Testing	Week 13 – Week 14	Unit Test Logs, Security Audit (SHA-256 validation), UAT Sign-off
Phase 6: Final Submission & Defense	Week 15	Deployed System, Final Documentation, Academic Defense Presentation

2. References

- Formal Standards
- IEEE Standard 830-1998: Recommended Practice for Software Requirements Specifications (SRS).
- Industry Benchmarks & Platforms
- Uber & Careem: Industry benchmarks used for real-time logistics, low-latency rental availability, fair/transparent pricing, and modern end-user booking experience.
- Pak car rentals: Existing paper/manual system reference.
- New Pakistan motors: Existing automotive rental market benchmark.
- PakWheels: Automotive marketplace and vehicle rental reference platform.
- Rent A Car Services 24/7: Online vehicle rental service reference (<https://www.rentacarservices247.com/>).

3. Definitions and Acronyms

- SPMP: Software Project Management Plan.
- SRS: Software Requirements Specification.
- ERD: Entity Relationship Diagram.
- RBAC: Role-Based Access Control (Admin, Registered Customer, Guest User).
- WBS: Work Breakdown Structure.
- Fleet Management: Administrative workflows for adding, updating status (available/under maintenance), or decommissioning vehicles.
- Three-Tier Architecture: Structural system model organizing software into Presentation Layer (HTML5/Bootstrap), Application Layer (Node.js/PHP), and Data Layer (MySQL).
- UAT: User Acceptance Testing.

4. Project Context and Technical Process Plans

4.1 Process Model

The project adopts the Agile Scrum Methodology, organizing engineering tasks into iterative, 2-week time-boxed sprints. Scrum allows continuous refinement of booking workflows, ongoing verification with academic supervisors, and early integration of complex external payment and mapping APIs

- Sprint 0 (Initiation & Backlog Definition): Setup version control on GitHub, finalize the SPMP, establish team roles, and refine product backlogs.
- Sprint 1 (Analysis & Structural Design): Complete the IEEE 830 SRS, model overall Use Cases, build the Normalized Database Schema, and finalize ERD relationships.
- Sprint 2 (Authentication & Profile Management): Build User Registration (11-digit phone number, input validation), Login sessions, and Password Reset flowcharts.
- Sprint 3 (Fleet Management & Catalog Engine): Build Admin controls for adding/updating vehicles and categories, along with customer-facing search filters (make, model, price, dates).
- Sprint 4 (Booking Engine & Payment Gateway): Implement automated rental duration calculation, date locking (preventing double-bookings), and credit card/Stripe integration.
- Sprint 5 (Quality Assurance & Hardening): Run unit test suites, conduct security audits (SHA-256 hashing, SQL injection checks), and execute User Acceptance Testing.
- Sprint 6 (System Sign-Off & Defense): Package final documentation, complete cloud deployment, and deliver the final presentation.

4.2 Methods, Tools, and Techniques

- Project Management: ProjectLibre (Gantt chart generation, milestone planning, resource tracking, and critical path analysis).
- Modeling & Architecture: Visual Paradigm / Lucidchart (UML Use Case, Activity diagrams, Process Flowcharts, and ERD data models).
- Version Control & Collaboration: Git and GitHub (structured feature branching, commit logs, and peer code reviews).
- Frontend Development: HTML5, CSS3 (Bootstrap framework for responsive grids), and modern JavaScript.
- Backend Development: Node.js (Express framework) or PHP for server-side logic and RESTful service routing.
- Database System: MySQL (relational storage with foreign key constraints, table normalization, and indexed queries).
- External Integration Services: Stripe/PayPal APIs (secure electronic card billing) and Google Maps API (spatial vehicle and location tracking).

4.3 Product Acceptance Plan

Formal sign-off requires meeting clear functional, architectural, and performance standards:

- Registration & Access Verification: Successful registration requiring a valid name, email, password (8+ characters), and an exact 11-digit phone number. Password reset flows must verify email records before allowing password updates.
- Catalog & Search Latency: Search queries based on category, date range, and model must return matching records in under 1.5 seconds.
- Booking & Overbooking Prevention: The booking engine must calculate total costs based on rental duration and car category, then block that vehicle from simultaneous conflicting bookings.
- Fleet Administrative Control: Administrators must be able to add vehicles, edit rental pricing, and update availability status (Active, Booked, Under Maintenance) with changes appearing immediately across user search results.

- **Security & Encryption:** Passwords and sensitive user data must be hashed using SHA-256 or equivalent standards before database insertion.

4.4 Product Acceptance Methods

1. **Unit Testing:** Automated unit testing of core calculation algorithms (e.g., date-difference duration calculation, total cost computation, phone number format verification) to confirm isolated function accuracy.
2. **Integration Testing:** End-to-end testing between the presentation layer, backend controllers, external payment gateway webhooks, and the MySQL database.
3. **User Acceptance Testing (UAT):** Real-world scenario testing with end-users and administrators running through customer booking, online payment, password recovery, and fleet management workflows.

5. Project Work and Schedule Allocation

5.1 Project Work Plan (WBS Structure)

- 1.0 Project Initiation & Planning
 - 1.1 Formulate SPMP Document (ISO/IEC/IEEE 16326)
 - 1.2 Initialize GitHub Project Repository & Branching Strategy
 - 1.3 ProjectLibre Gantt Chart & Schedule Baseline Creation
- 2.0 Requirements Engineering & Modeling
 - 2.1 System Scope & Literature Review (Uber, Careem, Indrive Analysis)
 - 2.2 Functional & Non-Functional Requirements Specification
 - 2.3 UML Diagram Modeling (Use Case Diagrams, Activity Flows)
- 3.0 System Architecture & Database Design
 - 3.1 Normalized MySQL Relational Schema Design
 - 3.2 Entity Relationship Diagram (ERD) Finalization
 - 3.3 UI/UX Responsive Layout Prototyping (Bootstrap Screens)
- 4.0 Implementation Sprints
 - 4.1 Sprint 2: RBAC, User Registration & Password Recovery Workflow
 - 4.2 Sprint 3: Vehicle Inventory Management (Admin Fleet CRUD)
 - 4.3 Sprint 3: Multi-Parameter Vehicle Search & Filtering Engine
 - 4.4 Sprint 4: Automated Cost Calculation & Reservation Engine
 - 4.5 Sprint 4: Payment Gateway API Integration (Card Billing)
- 5.0 Verification, Hardening & Deployment
 - 5.1 Execution of Unit & Functional Test Cases
 - 5.2 Security Validation (SHA-256 Hashing, Input Sanitization)
 - 5.3 User Acceptance Testing (UAT) Execution
 - 5.4 Final Deployment, Project Packaging & Academic Defense

5.2 Work Activities

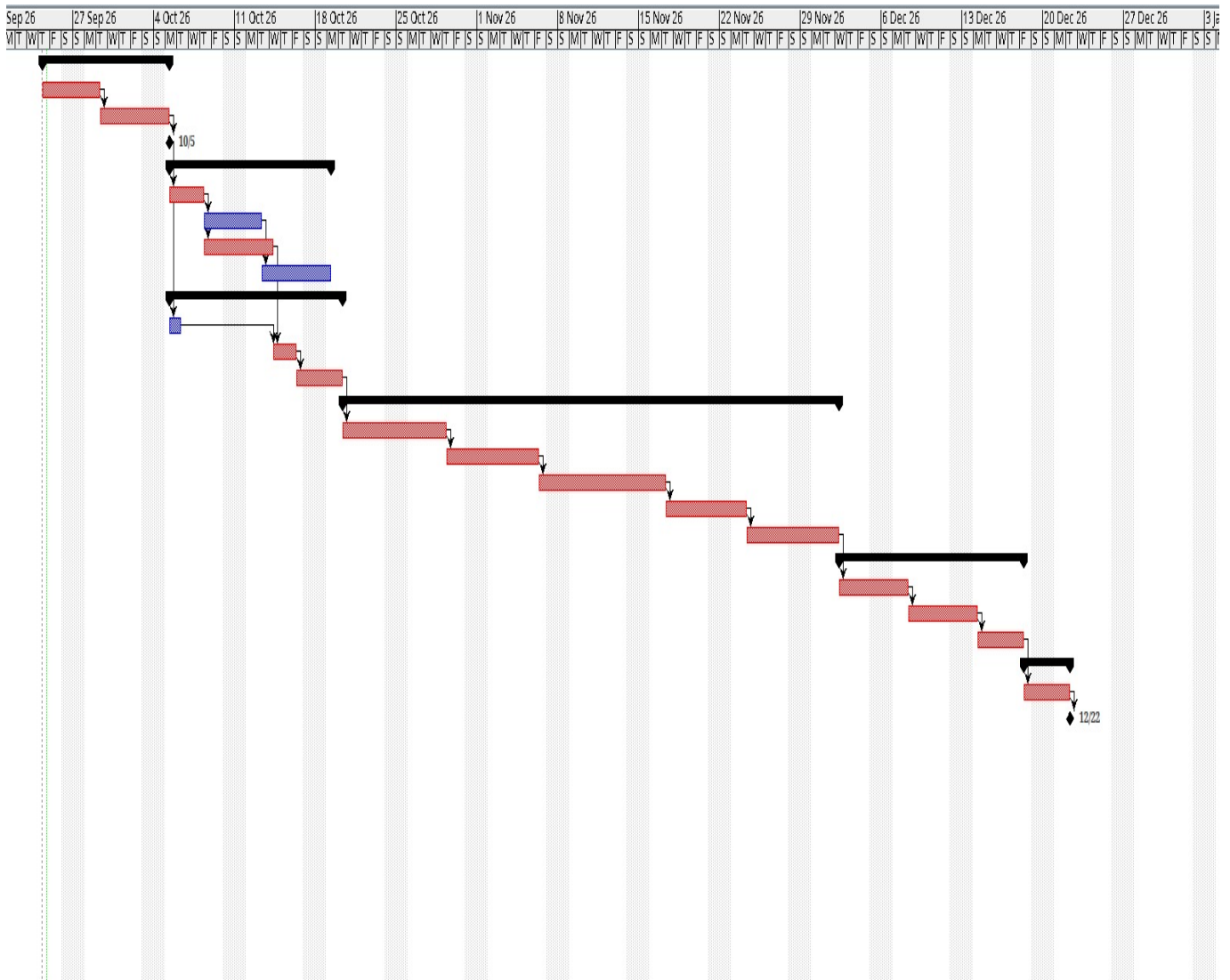
- **Weekly Progress & Sync Meetings:** Weekly standups to evaluate milestone progress, resolve architectural blockers, and track sprint backlogs.
- **Code Inspection & Peer Reviews:** Pull request reviews to ensure modular separation across the three tiers and maintain secure coding standards.
- **Traceability Verification:** Maintaining a Requirements Traceability Matrix linking Functional Requirements (FR001–FR004) directly to database tables, API routes, and test cases.

5.3 Schedule Allocation

The baseline project schedule spans a standard 15-week academic semester:

		Name	Duration	Start	Finish
1		☐1. Requirements & System Analysis	7 days	9/24/26, 9:00 AM	10/5/26, 9:00 AM
2		1.1 Requirements Elicitation & Scope Definition	3 days	9/24/26, 9:00 AM	9/29/26, 9:00 AM
3		1.2 Use Case Modeling & Flow of Events	4 days	9/29/26, 9:00 AM	10/5/26, 9:00 AM
4		1.3 SRS Documentation & Sign-off	0 days	10/5/26, 9:00 AM	10/5/26, 9:00 AM
5		☐2. System Design & Data Modeling	10 days	10/5/26, 9:00 AM	10/19/26, 9:00 AM
6		2.1 Domain & Data Flow Diagrams (DFD L0, L1, L2)	3 days	10/5/26, 9:00 AM	10/8/26, 9:00 AM
7		2.2 System Sequence & Activity Diagrams	3 days	10/8/26, 9:00 AM	10/13/26, 9:00 AM
8		2.3 Database Schema Design & ERD Modeling	4 days	10/8/26, 9:00 AM	10/14/26, 9:00 AM
9		2.4 UI Wireframing & Interface Mind Mapping	4 days	10/13/26, 9:00 AM	10/19/26, 9:00 AM
10		☐3. Architecture & Environment Setup	11 days	10/5/26, 9:00 AM	10/20/26, 9:00 AM
11		3.1 Repository, Git & Branching Strategy Setup	1 day	10/5/26, 9:00 AM	10/6/26, 9:00 AM
12		3.2 Database Initialization & Table Creation	2 days	10/14/26, 9:00 AM	10/16/26, 9:00 AM
13		3.3 Server & Backend Scaffold Setup	2 days	10/16/26, 9:00 AM	10/20/26, 9:00 AM
14		☐4. Development (Sprint Cycle)	31 days	10/20/26, 9:00 AM	12/2/26, 9:00 AM
15		4.1 Sprint 1: Auth & Customer Profile	7 days	10/20/26, 9:00 AM	10/29/26, 9:00 AM
16		4.2 Sprint 2: Vehicle Search & Catalog	6 days	10/29/26, 9:00 AM	11/6/26, 9:00 AM
17		4.3 Sprint 3: Reservation & Dynamic Pricing	7 days	11/6/26, 9:00 AM	11/17/26, 9:00 AM
18		4.4 Sprint 4: Payments & External APIs	5 days	11/17/26, 9:00 AM	11/24/26, 9:00 AM
19		4.5 Sprint 5: Admin Module & Reporting	6 days	11/24/26, 9:00 AM	12/2/26, 9:00 AM
20		☐5. Quality Assurance & Testing	12 days	12/2/26, 9:00 AM	12/18/26, 9:00 AM
21		5.1 Unit Testing (Pricing logic, validation)	4 days	12/2/26, 9:00 AM	12/8/26, 9:00 AM
22		5.2 Integration Testing (API, Database, Frontend)	4 days	12/8/26, 9:00 AM	12/14/26, 9:00 AM
23		5.3 User Acceptance Testing (UAT) & Test Cases	4 days	12/14/26, 9:00 AM	12/18/26, 9:00 AM
24		☐6. Final Deployment & Project Sign-Off	2 days	12/18/26, 9:00 AM	12/22/26, 9:00 AM
25		6.1 Production Server Deployment	2 days	12/18/26, 9:00 AM	12/22/26, 9:00 AM
26		6.2 Project Sign-off & Final Presentation	0 days	12/22/26, 9:00 AM	12/22/26, 9:00 AM

5.4 Resource Allocation



6. Supporting Process Plans

6.1 Risk Management

6.1.1 Risk Identification

- Risk R-01 (Technical - Double-Booking Concurrency): Simultaneous booking attempts for the same vehicle during overlapping date ranges, causing reservation conflicts.
- Risk R-02 (Technical - External Payment Gateway Sandbox Latency): Delays or connection drops from external payment gateway APIs during transaction handshakes.
- Risk R-03 (Technical - Data Integrity & Security Breaches): Unencrypted storage of customer records or passwords, exposing sensitive information.
- Risk R-04 (Technical - Search Query Latency): Complex SQL joins across large vehicle and booking tables exceeding the 1.5-second response time limit.
- Risk R-05 (Operational - Schedule Slippage): Midterm exams and academic deadlines delaying planned sprint deliverables.

6.1.2 Risk Analysis and Prioritization

Risk ID	Description	Category	Probability	Impact	Severity Level	
R-01	Vehicle double-booking under concurrency	Technical	Medium	High	High	
R-02	External payment gateway API failures	Technical	Medium	High	High	
R-03	Exposure of unencrypted passwords / user data	Technical	Low	High	High	
R-04	Search query response exceeding 1.5 seconds	Technical	Medium	Medium	Medium	
R-05	Academic schedule conflicts and project delays	Operational	High	Medium	High	

6.2 Risk Mitigation Strategies

- Mitigation for R-01 (Double-Booking Concurrency): Enforce ACID-compliant transactional locks in MySQL during checkout; check date availability within an isolated transaction before writing the booking record.
- Mitigation for R-02 (Payment Gateway Failures): Implement fallback routes, strict timeout handlers, and manual payment verification options if third-party endpoints become unresponsive.
- Mitigation for R-03 (Security & Password Exposure): Hash all user passwords using salted SHA-256 or bcrypt prior to database storage. Enforce input sanitization to prevent SQL injection vulnerabilities.
- Mitigation for R-04 (Search Latency): Index key columns in MySQL (e.g., 'car_type', 'availability_status', 'model') and use normalized tables to keep search query execution times below 1.5 seconds.
- Mitigation for R-05 (Schedule Slippage): Build a 1-week buffer into the schedule before final delivery, identify critical path items using ProjectLibre, and redistribute tasks across team members if delays occur.

6.3 Risk Monitoring and Control

- Sprint-End Risk Reviews: Review and update the project risk register at the close of every 2-week sprint cycle.
- Shared Defect Ledger: Log all identified software bugs, security flaws, and failed test cases in a centralized tracking sheet to ensure resolution prior to deployment.
- Supervisory Milestone Check-ins: Conduct bi-weekly progress reviews with the project supervisor to ensure engineering deliverables remain aligned with academic and institutional expectations.